Realising, Visualising and Applying Quantum Gates - Derivations

Will Noble

June 2026

1 An Important Message

These notes have been provided as a supplementary resource for my poster - Realising, Visualising and Applying Quantum Gates. This is purposefully calculation heavy, with possibly trivial knowledge being explained in full for the sake of completeness, such as the inclusion of trigonometric formulas that you see during A Levels.

Text highlighted in Red has been highlighted to present important results, most likely being seen within the poster, such as the table of realised single qubit gates.

There is also supplementary Python code, written with the following packages: NumPy, Matplotib, QuTip, CUDA-Q, SciPy, ImageIO, OS and Qiskit. This has been used to numerically confirm results, and has been featured in some chapters in order to justify reasoning.

THIS IS NOT TO BE A PUBLISHED PAPER, SOME CITATIONS ARE MISSING AND THUS SHOULD NOT BEEN SEEN AS A COMPLETE DOCUMENT, THIS IS ENTIRELY SUPPLEMENTARY FOR THE POSTER, WHICH HAS BEEN CITED CORRECTLY, SEEN VIA Citations_for_Poster.pdf.

2 Prerequisites

We must initially define the Pauli matrices.

$$\sigma_X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad \sigma_Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \qquad \sigma_Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

It is important to understand what a Quantum state $|\phi\rangle$ or $\langle\phi|$ is. We define $|\phi\rangle$ as a column vector representation, and $\langle\phi|$ as a row vector representation. This becomes important to know when we consider the solutions to the Time-dependent Schrödinger equation, as it only makes sense to multiply vectors / matrices in the order (2x2) $\cdot$ (2x1).

For our 1-qubit gates, we should define $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$. This is the orthonormal basis for $\mathbb{R}^2$ and classically, we would define $|0\rangle$ and $|1\rangle$ as results of a coin flip, with them representing heads and tails respectively. It is quite useful to know that qubits are very similar to classical bits, yet they can be states other than 0 and 1, such as $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$.

Some additional rules to these states, which we explain in the 2-qubit case in order to get the idea:

$|\phi\rangle = \alpha_1|00\rangle + \alpha_2|01\rangle + \alpha_3|10\rangle + \alpha_4|11\rangle$

With $\sum_i |\alpha_i|^2 = 1$

Where $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ is the orthonormal basis for a 2-qubit system.

It might be useful to know that $|0\rangle \otimes |0\rangle = |00\rangle$, where $\otimes$ is the tensor product and the result $|00\rangle = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$.

Generally we can define $\begin{pmatrix} a \\ b \end{pmatrix} \otimes \begin{pmatrix} c \\ d \end{pmatrix} = \begin{pmatrix} a \begin{pmatrix} c \\ d \end{pmatrix} \\ b \begin{pmatrix} c \\ d \end{pmatrix} \end{pmatrix}$.

This also applies for matrices in the same way, and the use of the tensor product will become useful when deriving a new definition for the Quantum Fourier Transform.

From our definitions above, the size of an n-qubit state is $N = 2^n$. This links to the dimension of Hilbert Spaces, which will not be discussed here.

We also define the Exact Hamiltonian. This has a physical representation corresponding to a driven two-level system, where:

ϵ: Energy level spacing

ω: Driving frequency

γ: Coupling strength

$$\mathcal{H}_{exact}(t) = \frac{\epsilon}{2}\sigma_Z + \gamma\sigma_X cos(\omega t)$$

It must be noted that $\hbar = 1$ for everything in this derivation and the poster.

Finally, we want to define the Time-dependent Schrödinger equation:

$$i\hbar\frac{\partial}{\partial t}|\psi(t)\rangle = \mathcal{H}(t)|\psi(t)\rangle$$

We can obtain the following solution from the Time-dependent Schrödinger equation:

$$|\psi(t)\rangle = e^{-i\mathcal{H}}|\psi(0)\rangle$$

We must recognise that this doesn't necessarily work for a time-dependent Hamiltonian, yet we will not be using this apart from comparing two solutions later on.

For a Time-dependent Hamiltonian, the following definition makes more sense:

$$|\psi(t)\rangle = \mathcal{T}exp(-i\int_0^t \mathcal{H}(t')dt')|\psi(0)\rangle$$

Where $\mathcal{T}$ is the Time Ordering operator. This is derived via a Dyson Series, yet we will not look at explaining this too complicated for the scope of this project. This is dropped for RWA as the RWA is time-independent.

3 Obtaining the Rotation Adjusted Hamiltonian

We start with $\mathcal{H}_{exact}(t) = \frac{\epsilon}{2}\sigma_Z + \gamma\sigma_X cos(\omega t)$ and $i\hbar\frac{\partial}{\partial t}|\psi(t)\rangle = \mathcal{H}(t)|\psi(t)\rangle$ as defined before.

We want to use $|\psi'(t)\rangle = e^{\frac{i\sigma_Z \omega t}{2}}|\psi(t)\rangle$ which is the rotating frame for ω.

We say that $\mathcal{R}(t) = e^{\frac{i\sigma_Z \omega t}{2}}$.

Apply the product rule and obtain $\frac{\partial}{\partial t}|\psi'(t)\rangle = \frac{\partial \mathcal{R}}{\partial t}|\psi(t)\rangle + \mathcal{R}\frac{\partial}{\partial t}|\psi(t)\rangle$.

Multiply by i to obtain $i\frac{\partial}{\partial t}|\psi'(t)\rangle = i\frac{\partial \mathcal{R}}{\partial t}|\psi(t)\rangle + i\mathcal{R}\frac{\partial}{\partial t}|\psi(t)\rangle$.

We substitute the rotating frame adjustment and Schrödinger equation for $\mathcal{H}_{exact}$ to obtain $i\frac{\partial}{\partial t}|\psi'(t)\rangle = i\frac{\partial \mathcal{R}}{\partial t}\mathcal{R}^{-1}|\psi'(t)\rangle + \mathcal{R}\mathcal{H}_{exact}|\psi(t)\rangle$.

$i\frac{\partial}{\partial t}|\psi'(t)\rangle = (i\frac{\partial \mathcal{R}}{\partial t}\mathcal{R}^{-1} + \mathcal{R}\mathcal{H}_{exact}\mathcal{R}^{-1})|\psi'(t)\rangle$ from substituting the rotating frame adjustment again.

We can now say $\mathcal{H}_{rot} = i\frac{\partial \mathcal{R}}{\partial t}\mathcal{R}^{-1} + \mathcal{R}\mathcal{H}_{exact}\mathcal{R}^{-1}$.

4 Obtaining the Rotating Wave Approximation Hamiltonian

$\mathcal{R}^{-1} = e^{\frac{-i\omega\sigma_Z t}{2}}$, $\frac{\partial R}{\partial t} = \frac{i\omega\sigma_Z}{2}\mathcal{R}$

Substituting these two results obtains $\mathcal{H}_{rot} = \frac{-\omega}{2}\sigma_Z + \mathcal{R}\mathcal{H}_{exact}\mathcal{R}^{-1}$.

Recalling $\mathcal{H}_{exact}(t) = \frac{\epsilon}{2}\sigma_Z + \gamma\sigma_X\cos(\omega t)$, we can substitute this into $\mathcal{R}\mathcal{H}_{exact}\mathcal{R}^{-1}$.

$\mathcal{R}\mathcal{H}\mathcal{R}^{-1} = \frac{\epsilon}{2}\mathcal{R}\sigma_Z\mathcal{R}^{-1} + \gamma\cos(\omega t)\mathcal{R}\sigma_X\mathcal{R}^{-1}$

What is $\mathcal{R}$ as a matrix?

$\mathcal{R}(t) = e^{\frac{i\sigma_Z\omega}{2}} = I + \frac{i\sigma_z\omega}{1!}t + \frac{(i\sigma_Z\omega)^2}{2!}t^2 + ...$ using the Taylor series for e^x.

Recall $\sigma_Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$

${\sigma_Z}^2 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = I$

So $\mathcal{R}(t) = (I - \frac{{\sigma_Z}^2}{2!}(\frac{\omega t}{2})^2 + ...) + i(\frac{\sigma_Z}{1!}\frac{\omega t}{2} - \frac{{\sigma_Z}^3}{3!}(\frac{\omega t}{2})^3 + ...)$

Recalling the Taylor series for $\sin(x)$ and $\cos(x)$:

$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - ...$

$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - ....$

This means $\mathcal{R}(t) = I\cos(\frac{\omega t}{2}) + i\sigma_Z\sin(\frac{\omega t}{2})$

And thus $\mathcal{R}^{-1}(t) = I\cos(\frac{\omega t}{2}) - i\sigma_Z\sin(\frac{\omega t}{2})$

$\sigma_Z\mathcal{R}^{-1} = \sigma_Z\cos(\frac{\omega t}{2}) - iI\sin(\frac{\omega t}{2})$

$\mathcal{R}\sigma_Z\mathcal{R}^{-1} = (I\cos(\frac{\omega t}{2}) + i\sigma_Z\sin(\frac{\omega t}{2}))(\sigma_Z\cos(\frac{\omega t}{2}) - iI\sin(\frac{\omega t}{2}))$

$\mathcal{R}\sigma_Z\mathcal{R}^{-1} = \sigma_Z\cos^2(\frac{\omega t}{2}) - iI^2\cos(\frac{\omega t}{2})\sin(\frac{\omega t}{2}) + i\sigma_Z^2\sin(\frac{\omega t}{2})\cos(\frac{\omega t}{2}) + \sigma_Z\sin^2(\frac{\omega t}{2})$

$\mathcal{R}\sigma_Z\mathcal{R}^{-1} = \sigma_z(\cos^2(\frac{\omega t}{2}) + \sin^2(\frac{\omega t}{2}))$

$\mathcal{R}\sigma_Z\mathcal{R}^{-1} = \sigma_Z$ using $\sin^2(\theta) + \cos^2(\theta) = 1$

There is probably an easier way to do this, but it's the first way I found, ignoring any potential commutativity of the matrices.

Now we have $\mathcal{H}_{rot}(t) = \frac{\epsilon-\omega}{2}\sigma_z + \gamma\cos(\omega t)\mathcal{R}\sigma_X\mathcal{R}^{-1}$

Now we want to find $\mathcal{R}\sigma_X\mathcal{R}^{-1}$.

$\sigma_X\mathcal{R}^{-1} = \sigma_X\cos(\frac{\omega t}{2}) - i\sigma_X\sigma_Z\sin(\frac{\omega t}{2})$

$\mathcal{R}\sigma_X\mathcal{R}^{-1} = (I\cos(\frac{\omega t}{2}) + i\sigma_Z\sin(\frac{\omega t}{2}))(\sigma_X\cos(\frac{\omega t}{2}) - i\sigma_X\sigma_Z\sin(\frac{\omega t}{2}))$

$\mathcal{R}\sigma_X\mathcal{R}^{-1} = \sigma_X\cos^2(\frac{\omega t}{2}) - i\sigma_X\sigma_Z\sin(\frac{\omega t}{2})\cos(\frac{\omega t}{2}) + i\sigma_Z\sigma_X\sin(\frac{\omega t}{2})\cos(\frac{\omega t}{2}) + \sigma_Z\sigma_X\sigma_Z\sin^2(\frac{\omega t}{2})$

We must calculate the matrices $-i\sigma_X\sigma_Z$, $i\sigma_Z\sigma_X$ and $\sigma_Z\sigma_X\sigma_Z$.

$$-i\sigma_X\sigma_Z = -i\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix}\begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix} = -i\begin{pmatrix}0 & -1\\ 1 & 0\end{pmatrix} = -\sigma_Y$$
$$i\sigma_Z\sigma_X = i\begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix}\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix} = i\begin{pmatrix}0 & 1\\ -1 & 0\end{pmatrix} = -\sigma_Y$$
$$\sigma_Z\sigma_X\sigma_Z = \begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix}\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix}\begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix} = -\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix} = -\sigma_X$$

Now we can find $\mathcal{R}\sigma_X\mathcal{R}^{-1}$.

$\mathcal{R}\sigma_X\mathcal{R}^{-1} = \sigma_X\cos^2(\frac{\omega t}{2}) - \sigma_X\sin^2(\frac{\omega t}{2}) - 2\sigma_Y\sin(\frac{\omega t}{2})\cos(\frac{\omega t}{2})$

$\mathcal{R}\sigma_X\mathcal{R}^{-1} = \sigma_X(\cos^2(\frac{\omega t}{2}) - \sin^2(\frac{\omega t}{2})) - \sigma_Y(2\sin(\frac{\omega t}{2})\cos(\frac{\omega t}{2}))$

Double Angle Formulae:

$\cos(2\theta) = \cos^2(\theta) - \sin^2(\theta)$

$\sin(2\theta) = 2\sin(\theta)\cos(\theta)$

So $\mathcal{R}\sigma_X\mathcal{R}^{-1} = \sigma_X\cos(\omega t) - \sigma_Y\sin(\omega t)$.

Now $\gamma\cos(\omega t)\mathcal{R}\sigma_X\mathcal{R}^{-1} = \gamma\sigma_X\cos^2(\omega t) - \gamma\sigma_Y\cos(\omega t)\sin(\omega t)$

$\cos(2\theta) = 2\cos^2(\theta) - 1$ and thus $\cos^2(\theta) = \frac{\cos(2\theta)+1}{2}$ from Double Angle Formulae

So $\gamma\cos(\omega t)\mathcal{R}\sigma_X\mathcal{R}^{-1} = \frac{\gamma}{2}\sigma_X + \frac{\gamma}{2}\sigma_X\cos(2\omega t) - \frac{\gamma}{2}\sigma_Y\sin(2\omega t)$

Now $\mathcal{H}_{rot}(t) = \frac{1}{2}(\epsilon - \omega)\sigma_Z + \frac{\gamma}{2}\sigma_X + V(t)$

Where $V(t) = \frac{\gamma}{2}(\sigma_X\cos(2\omega t) - \sigma_Y\sin(2\omega t))$

When we drop the V(t) from $\mathcal{H}_{rot}$, you obtain $\mathcal{H}_{RWA}$.

Thus we obtain the time independent $\mathcal{H}_{RWA} = \frac{1}{2}(\epsilon - \omega)\sigma_Z + \frac{\gamma}{2}\sigma_X$

5 Why can we drop V(t)?

We very suddenly dropped the time dependent part of $\mathcal{H}_{rot}$, the $V(t)$. Why do we want to do this?

The motivation for this is to have a time independent Hamiltonian, such that time leaves it unaffected when we try and tune parameters for logic gates.

In order to see whether this is a viable change, we must compare the solutions of the Time-dependent Schrödinger equation. These have been calculated in Python using the SciPy package.

```python
def inner_product(f, g, frame_transform_f=False, frame_transform_g=False):
    """
    This function will compute the inner product of 2 functions, which helps us measure the overlap of the functions.
    It is often seen by the notation <f,g>. For our examples, the inner product will be defined on [0,T]. The T will be our time interval in which we solved the ODE earlier.
    If you want to use an exact Hamiltonian, you will want to set frame_transform to the function (set to True) in order for everything to be in a rotating frame.
    """

    fidelity = []
    for i in range(f.shape[1]):
        ψ_f = f[:, i] # This rotates the exact function into the rotating frame
        ψ_g = g[:, i]
        if frame_transform_f is True:
            rot_adj_f = rotating_frame_adjustment(t_axis[i])
            ψ_f = rot_adj_f @ ψ_f
            # This ensures we get the ψ' that we need
        if frame_transform_g is True:
            rot_adj_g = rotating_frame_adjustment(t_axis[i])
            ψ_g = rot_adj_g @ ψ_g
        overlap = np.vdot(ψ_f, ψ_g) # For vdot, check https://numpy.org/doc/stable/reference/generated/numpy.vdot.html
        fidelity.append(np.abs(overlap)**2)
    fidelity = np.array(fidelity)
    average_fidelity = trapezoid(fidelity, t_axis) / (t_axis[-1] - t_axis[0]) # This computes an average
    fidelityerror = 1 - min(fidelity)
    return average_fidelity, fidelityerror
```

Figure 1: Inner Product Function

We use the idea of Fidelity. "Fidelity in quantum computing measures the accuracy of quantum operations – essentially how closely real quantum processes match the ideal, error-free processes."(Ivezic 2023)

We find that for the following tuples (in the order $(\epsilon, \gamma, \omega)$), we obtain the following Fidelity and Max Fidelity Error:
(11, 1, 10), 0.9980862569699225, 0.006619300129882544
(41, 1, 40), 0.9998696553789628, 0.000448888096092781
(1001, 1, 1000), 0.9999995285788046, 1.244694879054542e-06

The following tuples were used to respect an inequality mentioned in Section 5, and these are seen in Lecture 2 (Barnett 2026, p. 17).

A Fidelity of 0.99 or greater is seen as optimal, and it can be seen that this is far exceeded by the Rotating Wave Approximation Hamiltonian when compared to the Exact Hamiltonian in the solution to the Schrödinger equation.

However, there's a slight hitch in our justification. We've only chosen a small

T, T = 10 in the following calculations:

$$\bar{F} = \frac{1}{T}\int_0^T F(t)dt, \qquad F(t) = |\langle\psi_{rot}(t)|\psi_{RWA}(t)\rangle|^2, \qquad 0 \leq \bar{F} \leq 1, T \in \mathbb{R}_{\geq 0}$$

We've used a small T, and thus our justification isn't actually all that strong. If we pick $T = 1000$, our justification actually breaks down for a small tuple (11,1,10). However, for (1001,1,1000), we obtain 0.998675399136747, 0.003971685201408226, our respective Fidelity and Maximum Fidelity error results, so we get the idea that as we increase T, we should increase our ϵ, γ and ω.

It seems like a good idea to find values of F (Fidelity) and graph it against values of T, in order to see whether our assumption holds for large T, but this is computationally very time consuming. Instead, we consider the following integral:

$$\int_0^T V(t)dt \approx 0$$

This is the time-average of $V(t)$

We now want to calculate: $\int_0^T \frac{\gamma}{2}(\sigma_X \cos(2\omega t) - \sigma_Y \sin(2\omega t))dt$

Which is $[\frac{\gamma}{2}(\sigma_X \frac{\sin(2\omega t)}{2\omega} + \sigma_Y \frac{\cos(2\omega t)}{2\omega})]$ with t = 0, t = T.

This results in $\frac{\gamma}{2}(\sigma_X \frac{\sin(2\omega T)}{2\omega} + \sigma_Y \frac{\cos(2\omega T)}{2\omega} - \sigma_Y \frac{1}{2\omega})$

If we take $\lim_{\omega\to\infty}$ with $\lim_{T\to\infty}$ and we know that $|\sin(x)| \leq 1$ and $|\cos(x)| \leq 1$ $\forall x$, then we find that $\int_0^T V(t)dt$ will tend to 0.

What we can deduce for this is for a large T, in order to achieve the Rotating Wave Approximation and drop $V(t)$, we also want to achieve a sufficiently large ω.

Of course, we still want $\sqrt{(\epsilon-\omega)^2 + \gamma^2} \ll \omega$, so ϵ and γ must also be picked accordingly.

Finally, this isn't actually the end of the story for $V(t)$. There are other edge cases which lead to the RWA failing, but these will not stop our realisation of quantum gates, for now.

6 Deriving the Hadamard and Phase Gate

Realised gates have the form $U(\tau) = e^{-iH_{RWA}\tau}$. A unitary operator, the propagator, can be used to derive this result.

We must have the following inequality hold as well $\sqrt{(\epsilon-\omega)^2+\gamma^2} \ll \omega$.

For convenience, $\Delta = \sqrt{(\epsilon-\omega)^2+\gamma^2}$.

We compare these to our Sought gate, $H = \frac{1}{\sqrt{2}}(\sigma_X + \sigma_Z)$ and $T = e^{-i\phi\sigma_Z}$.

We start with T, the Phase Gate.

Comparing U and T, $\phi\sigma_Z = \mathcal{H}_{RWA}\tau$.

We know $\mathcal{H}_{RWA} = \frac{1}{2}(\epsilon-\omega)\sigma_Z + \frac{\gamma}{2}\sigma_X$.

We must pick $\gamma = 0$ and $\epsilon \neq \omega$, with $\epsilon \ll \omega$.

Comparing $\frac{1}{2}(\epsilon-\omega)\tau = \phi$, we get $\tau = \frac{2\phi}{\epsilon-\omega}$.

So the Phase Gate is realised for $\tau = \frac{2\phi}{\epsilon-\omega}, \gamma = 0$ and $\epsilon \neq \omega$.

We want to find the Hadamard gate now.

Ideally, we want to represent $U(\tau) = e^{-iH_{RWA}\tau}$ as a function of matrices, not as an matrix exponential.

$U(\tau) = I - i\frac{\mathcal{H}_{RWA}\tau}{1!} - \frac{(\mathcal{H}_{RWA}\tau)^2}{2!} + i\frac{(\mathcal{H}_{RWA}\tau)^3}{3!} + ...$

$U(\tau) = (I - \frac{(\mathcal{H}_{RWA}\tau)^2}{2!} + ...) - i(\frac{\mathcal{H}_{RWA}\tau}{1!} - \frac{(\mathcal{H}_{RWA}\tau)^3}{3!} + ...)$

It makes sense to ask what ${\mathcal{H}_{RWA}}^2$ is:

${\mathcal{H}_{RWA}}^2 = (\frac{1}{2}(\epsilon-\omega)\sigma_Z + \frac{\gamma}{2}\sigma_X)(\frac{1}{2}(\epsilon-\omega)\sigma_Z + \frac{\gamma}{2}\sigma_X)$

${\mathcal{H}_{RWA}}^2 = \frac{1}{4}(\epsilon-\omega)^2{\sigma_Z}^2 + \frac{\gamma}{4}\sigma_Z\sigma_X + \frac{\gamma}{4}\sigma_X\sigma_Z + \frac{\gamma^2}{4}{\sigma_X}^2$

We only need to check ${\sigma_X}^2$.

${\sigma_X}^2 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I$

So ${\mathcal{H}_{RWA}}^2 = \frac{1}{4}(\epsilon-\omega)^2 I + \frac{i\gamma}{4}\sigma_Y - \frac{i\gamma}{4}\sigma_Y + \frac{\gamma^2}{4}I$

```python
def global_phase_comparison(U, V, tol=1e-8):
    """
    This function takes in 2 matrices that you want to compare the properties of, disregards the global phase difference.
    """
    overlap = np.vdot(U.flatten(), V.flatten()) # Flatten changes the matrices to a 4x1 column vector
    if np.abs(overlap) < tol:
        return False # This checks whether the flattened matrices are orthogonal.
    phase = overlap / np.abs(overlap)
    return np.allclose(U, V / phase, atol=tol)
```

Figure 2: Global Phase Comparison

```python
if np.isclose(ε, ω):
    print("Cannot realise the Hadamard Gate for ε = ω.")
else:
    if (global_phase_comparison(hadamard, realised_func(np.pi/(np.sqrt(2) * (ε - ω))))):
        print("The Hadamard Gate has been realised for τ = π/(sqrt(2) * γ), γ = ε - ω.")
        print(f"The gate fidelity is {gate_fidelity(hadamard, realised_func(np.pi/(np.sqrt(2) * (ε - ω))))}.")
    else:
        print("The Hadamard Gate hasn't been realised due to a choice of τ and γ.")
        print(f"The gate fidelity is {gate_fidelity(hadamard, realised_func(np.pi/(np.sqrt(2) * (ε - ω))))}.")
```

Figure 3: Hadamard Phase Comparison, trivial example for $k = 1$

${\mathcal{H}_{RWA}}^2 = \frac{1}{4}((\epsilon - \omega)^2 + \gamma^2)I = \frac{\Delta^2}{4}I$

$U(\tau) = (I - \frac{1}{2!}(\frac{\tau\Delta}{2})^2 I + ...) - i(\frac{\mathcal{H}_{RWA}\tau}{1!} - \frac{(\mathcal{H}_{RWA}\tau)^3}{3!} + ...)$

$U(\tau) = (I - \frac{1}{2!}(\frac{\tau\Delta}{2})^2 I + ...) - \frac{2i\mathcal{H}_{RWA}}{\Delta}(\frac{1}{1!}(\frac{\tau\Delta}{2}) - \frac{1}{3!}(\frac{\tau\Delta}{2})^3 + ...)$

$U(\tau) = I\cos(\frac{\tau\Delta}{2}) - \frac{2i\mathcal{H}_{RWA}}{\Delta}\sin(\frac{\tau\Delta}{2})$ using Taylor series of $\cos(x)$ and $\sin(x)$.

We now want to substitute in Δ and $\mathcal{H}_{RWA}$.

$U(\tau) = I\cos(\frac{\tau\sqrt{(\epsilon-\omega)^2+\gamma^2}}{2}) - i(\frac{(\epsilon-\omega)\sigma_Z + \gamma\sigma_X}{(\sqrt{(\epsilon-\omega)^2+\gamma^2})})\sin(\frac{\tau\sqrt{(\epsilon-\omega)^2+\gamma^2}}{2})$

We have now obtained an easier form of $U(\tau)$ to work with.

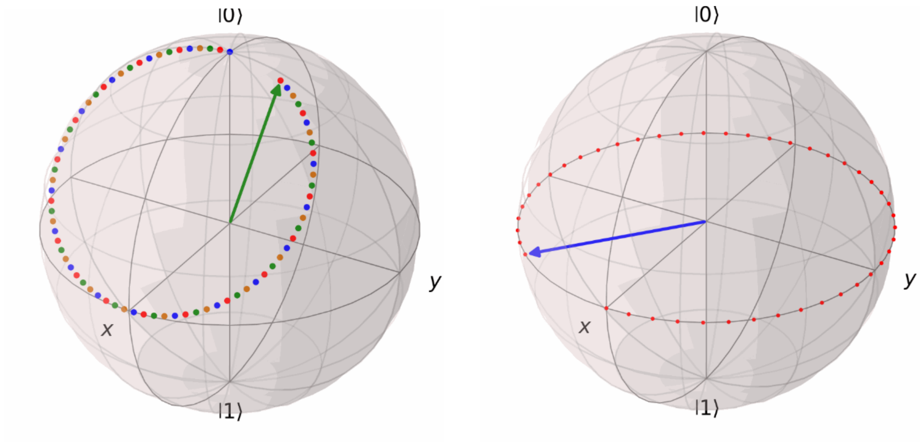
Considering the Hadamard Gate, $\frac{\tau\Delta}{2} = \frac{(2k+1)\pi}{2}, k \in \mathbb{Z}$ in order for $\cos(\frac{\tau\Delta}{2}) = 0$.

We find that $\tau = \frac{(2k+1)\pi}{\Delta}$ and $\gamma = \epsilon - \omega$ such that γ and $\epsilon - \omega$ are proportional.

This results in $\tau = \frac{(2k+1)\pi}{\sqrt{2}\gamma}$ and $\gamma = \epsilon - \omega$ after substitution. $\epsilon \neq \omega$.

It must be noted that we are left with an iH or iH, however this is a global phase difference that does not matter as the Quantum gate is functionally the same. Scalar differences of $e^{i\phi}$ are not significant on the realisation of the gate. This is mentioned by Nielsen and Chuang. (Nielsen and Chuang 2010, p. 93)

7 The Bloch Sphere



(a) Hadamard Gate

(b) Phase Gate

Figure 4: Bloch sphere representations of the gates.

Now we want to briefly look at a common representation of Quantum gates, in the form of the Bloch Sphere.

Recalling the Pauli matrices, σ_X, σ_Y and σ_Z, it can be determined that these result in π radians anticlockwise rotations in their respective axis after exponentiation if you consider a continuous time evolution. These do result in π radians anticlockwise rotations anyway, but not seen on the Bloch Sphere without exponentiation first.

Considering the Hadamard gate, applying H twice, as seen in Figure 4(a), must return the identity as the initial state $|0\rangle$ is the start and end of the trajectory. After 1 Hadamard gate, the state the qubit is in is $|+\rangle$.

We can confirm our expectation by evaluating H^2.

$$H^2 = \frac{1}{2}(\sigma_X + \sigma_Z)(\sigma_X + \sigma_Z)$$

$$H^2 = \frac{1}{2}({\sigma_X}^2 + \sigma_X \sigma_Z + \sigma_Z \sigma_X + {\sigma_X}^2)$$

$$H^2 = \frac{1}{2}(I + i\sigma_Y - i\sigma_Y + I)$$

$$H^2 = I$$

In the case of the Phase Gate, this is entirely a rotation about the Z-axis. If you were to start with $|0\rangle$ or $|1\rangle$, they would be unaffected by the Phase gate.

It is really important to recognise that the Hadamard gate is represented by a matrix, and does not display this time evolution in practice. This time evolution has been provided as a way of understanding how the gate acts on the Bloch Sphere, obtained via exponentiation.

8 Realising other 1-Qubit Gates

ALL GATES REALISED BELOW HAVE BEEN CHECKED IN PYTHON WITH THEIR DEFINITION. PYTHON FILE SUPPLEMENTED ELSEWHERE.

We've now obtained the Hadamard and Phase gates. From these, we can obtain other gates from these, or from the pre-defined Pauli Matrices. Using the substitution $\phi = \frac{\pi}{2}$, you can obtain the Phase S gate, and $\phi = \frac{\pi}{4}$, obtaining the $\frac{\pi}{8}$ gate. It should also be noted that $\phi = \pi$ will obtain the σ_Z gate, which we've already defined.

Before we go further, it is useful to realise that the Phase gate has a linear transformation representation as $P(\phi) = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\phi} \end{pmatrix}$. This is fundamentally the same to an alternative definition, found by representing the exponential sought form of the Phase gate as a matrix, and noticing that these definitions differ by a global phase which is irrelevant physically.

We now have 8 1-qubit gates, the X (NOT) Gate, Y, Z, S, Phase, Hadamard, $\frac{\pi}{8}$ and I, the Identity Gate (trivial result).

We can also realise the NOT gate from our earlier $U(\tau)$:

$U(\tau) = Icos(\frac{\tau\sqrt{(\epsilon-\omega)^2+\gamma^2}}{2}) - i(\frac{(\epsilon-\omega)\sigma_Z+\gamma\sigma_X}{(\sqrt{(\epsilon-\omega)^2+\gamma^2})})sin(\frac{\tau\sqrt{(\epsilon-\omega)^2+\gamma^2}}{2})$

Using $\epsilon = \omega, \gamma \neq 0$ and $\tau = \frac{(2k+1)\pi}{\gamma}, k \in \mathbb{Z}$, we obtain the X/NOT gate.

Trivially, $\epsilon = \omega, \gamma = 0$ results in the identity gate I.

Now let's consider the $\sqrt{X}$ gate. We will find out what $\sqrt{\sigma_X}$ is, which we will achieve by diagonalisation.

$\sigma_X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, $\chi_{\sigma_X}(x) = x^2 - 1$, such that its eigenvalues are $\lambda_1 = 1$ and $\lambda_2 = -1$.

With some calculations, $\vec{v_1} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ and $\vec{v_2} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$

Using the Diagonalisation formula seen in Linear Algebra and Groups - Term 2, we can obtain $\sigma_X = PDP^{-1}$, where:

$D = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$, $P = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$, $P^{-1} = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} \end{pmatrix}$

We now write $\sqrt{\sigma_X} = PD^{\frac{1}{2}}P^{-1}$, so $\sqrt{\sigma_X} = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix} \begin{pmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} \end{pmatrix}$. It is interesting to note here that $\sqrt{\sigma_X} = HSH$, seen by square rooting $\frac{1}{2}$, using these coefficients on P and P^{-1}.

Thus $\sqrt{\sigma_X} = \frac{1}{2}\begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix}$

We bring back $U(\tau) = e^{-iH_{RWA}\tau}$ and substitute our values for the X gate, such that $\sigma_X = e^{\frac{-i(2k+1)\pi\sigma_X}{2}}$, so we can square root both sides and obtain $\sqrt{\sigma_X} = e^{\frac{-i(2k+1)\pi\sigma_X}{4}}$.

We therefore pick $\epsilon = \omega, \gamma \neq 0$ and $\tau = \frac{(2k+1)\pi}{2\gamma}, k \in \mathbb{Z}$ achieving $\sqrt{X}$ gate.

We can now defined our 3 Rotation gates, with $R_X(\theta) = e^{\frac{-i\sigma_X\theta}{2}}$, $R_Y(\theta) = e^{\frac{-i\sigma_Y\theta}{2}}$ and $R_Z(\theta) = e^{\frac{-i\sigma_Z\theta}{2}}$.

We start with deriving $R_X(\theta)$ and $R_Z(\theta)$.

$\frac{\sigma_X\theta}{2} = (\frac{1}{2}(\epsilon - \omega)\sigma_Z + \frac{\gamma}{2}\sigma_X)\tau$

It is clear to see that $\epsilon = \omega$, now $\frac{\theta}{2} = \frac{\gamma\tau}{2}$, such that $\tau = \frac{\theta}{\gamma}$.

Overall, $R_X(\theta)$ is realised for $\epsilon = \omega, \tau = \frac{\theta}{\gamma}, \gamma \neq 0$.

The $R_Z(\theta)$ gate is fundamentally identical to the Phase gate, with a realisation that is very similar.

Now we define the $R_Y(\theta)$ gate as $SR_X(\theta)S^\dagger$.

I will also define $U(\theta, \phi, \lambda) = R_Z(\phi)R_Y(\theta)R_Z(\lambda)$, however the derivation for this will not be included in the document.

The two gates defined above are identities rather than definitions, but they will not be covered due to limitations of the $\mathcal{H}_{exact}$.

9 The Quantum Fourier Transform